



DC – AI – Routing – Switching – Security – Wireless – Automation

Let-s Talk About VOQ and DNX Pipeline

An HPE Juniper Networking Techpost

Nicolas Fevrier

2022-07-27

HPE



Contents

1	Let's Talk About VOQ and DNX Pipeline	3
1.1	Introduction	3
1.2	Ingress-only Buffering	3
1.3	Scheduled Forwarding and Virtual Output Queues	4
1.4	Life of a Packet	7
1.5	Illustration with CLI	10
1.6	The pipeline	19
1.7	Conclusion	20
1.8	Useful links	20
1.9	Glossary	20
1.10	Acknowledgements	21

List of Figures

1	Ingress-only buffering	3
2	Queues for egress port et-0/2/0	5
3	VOQ for two ports	6
4	example with 4 PFes and 4 egress interfaces	6
5	Life of a packet, Ingress	7
6	Life of a packet, Egress	8
7	Life of a packet in SoC architecture, Ingress	9
8	Life of a packet in SoC architecture, Egress	10
9	High-level view of the DNX Pipeline	19

1 Let's Talk About VOQ and DNX Pipeline

Nicolas Fevrier - 07/27/2022

To understand the life of a packet in an ACX7000 Series router, you first need to understand the idea behind Virtual Output Queues.

1.1 Introduction

Many Network Processing Unit (NPU) architectures are available on the market today. They propose different packet buffering approaches, performed in both ingress and egress datapaths (sometimes referred to as “2-stage buffering architecture”) or in the ingress pipeline only. That second case includes the DNX chipsets powering Juniper ACX Series. More details on the DNX options and their architecture has been covered in the first article of the series: <https://juniper.github.io/techposts/building-the-acx7000-series-the-pfe/article>

In this article, we describe this packet buffering logic by following the life of a packet, and we introduce the concept of a Virtual Output Queue (VOQ).

The concept described here will be useful for all follow up articles on ACX7k platforms since these principles are common to all products in this family.

1.2 Ingress-only Buffering

DNX ASICs used in ACX7000 Series are based on a pipeline design where the packet memory is present in the ingress part.

We talk about “ingress-only buffering model” but in all fairness, we should probably call it “ingress-mostly buffering”. Indeed, a small packet buffer is still present in the egress pipeline, but it’s a very shallow space compared to the very large High-Bandwidth Memory (HBM) used off-chip and reachable from the ingress pipeline.

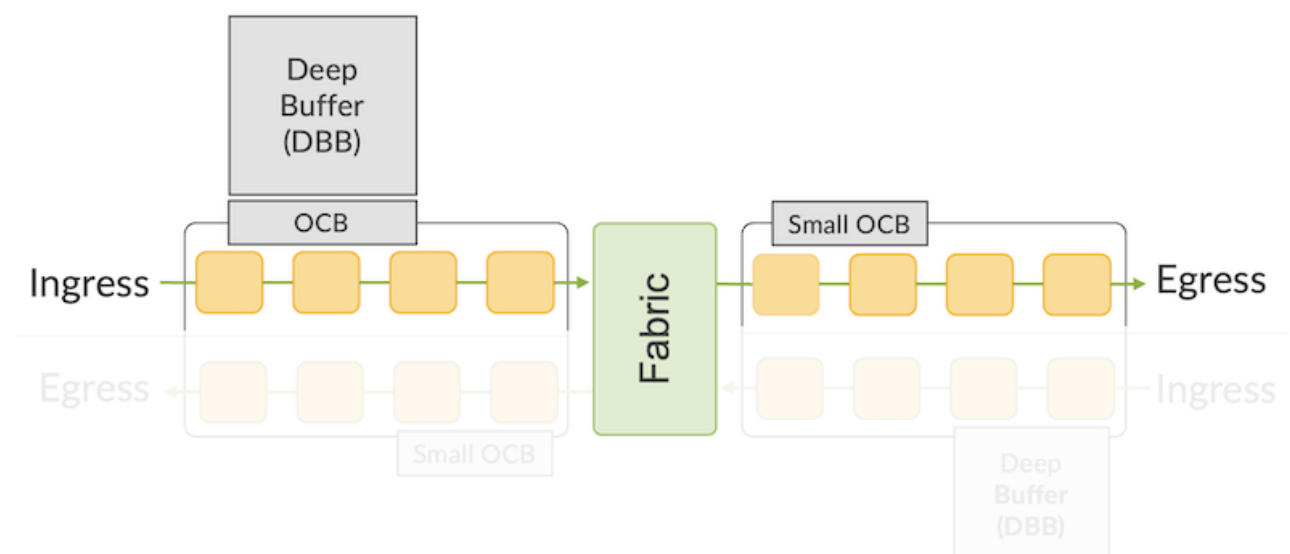


Figure 1: Ingress-only buffering

In normal traffic conditions, the packets are stored in the **ingress** On-Chip Buffer (OCB). It's only when a queue is getting congested that packets are moved to the Delay Bandwidth Buffer (DBB) or "Off-Chip Buffer". This second memory type will be an HBM or DDR.

This mechanism of queue eviction to an off-chip buffer and return to an on-chip buffer occurs dynamically as soon as a threshold is exceeded.

The small on-chip buffer on the **egress** pipeline can be used to store packets before they are sent to the interface and can only discriminate between high and low priority. It doesn't have the size nor the structure to perform Quality of Service (QoS) treatment.

To compare the different memory sizes, let's take the Jericho2 NPU example:

- The ingress on-chip buffer is 32 MB (16 per core)
- The ingress off-chip buffer is 8 GB (shared)
- The egress on-chip buffer is 12 MB (6 per core)

Using an ingress-only deep buffer offers multiple advantages:

- It reduces by half the memory requirement
- It reduces the footprint on the Printed Circuit Board (PCB)
- It reduces the latency
- It reduces power consumption
- Eventually, it optimizes the cost

Everything in technology is a trade-off, it creates other challenges. When the packets are stored in egress before transmission, it's easy to classify them and apply different treatments with QoS policies. But when packets are buffered in potentially dozens of ingress PFEs (Packet Forwarding Engines), we need a new mechanism to guarantee the forwarding efficiency. It requires fast communication between the ingress pipeline and its egress counterpart. We are talking about "scheduling".

1.3 Scheduled Forwarding and Virtual Output Queues

How does it work then?

When a packet is received in a port, it starts its journey in the ingress pipeline. A lookup determines the packet destination, and a classification associates the traffic to a queue. But again, the queues are not present in the outbound path, so we need to create a virtual representation of this pair (destination port, queue) that can be associated with every packet stored in the ingress pipeline.

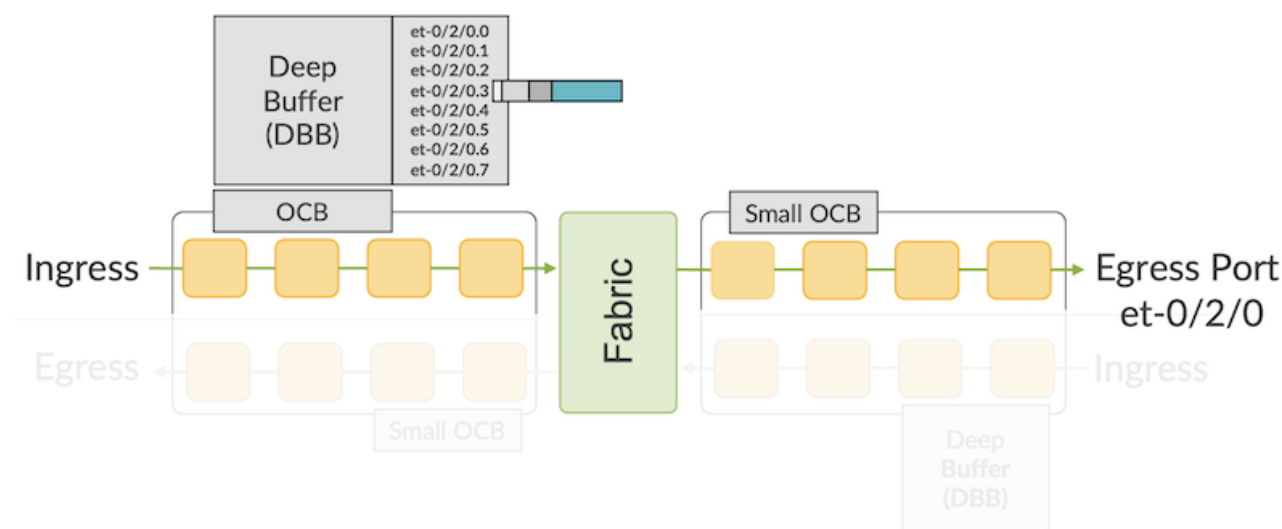


Figure 2: Queues for egress port et-0/2/0

In Diagram 2 above, we represent the queues created, from the ingress pipeline perspective, for port et-0/2/0. Note that it's not necessarily limited to a physical port, we can create "attach points" for these queues and it could be an IFL (logical interface, like a sub-interface for example).

When packets are received:

- Destination lookup is done, itself "resolved" to a destination interface (or again, an attachment point)
- Classification will be performed to differentiate packet class, by default or dictated by a QoS policy, eventually associating the packet to a queue.

At the end of the operation, we can store the packet in the buffer (for the sake of simplicity, let's imagine it will be done in the off-chip buffer, but the same logic applies to on-chip). The system stores the packet with an internal header representing the pair (destination port, queue) represented by a number. This number is a Virtual Output Queue Identifier (VOQ ID). Again, I insist on this important aspect, the queues are not carved in memory. It should be seen as a flag added to the packet.

Please don't write VoQ. The "O" is for "Output", not "of" (like in QoS). Ok, it's not that important, I agree.

To put it differently, a VOQ is the virtual representation of a queue and an egress port, but from the ingress pipeline perspective, where the buffering is actually done.

This representation exists for the port itself, as presented in the example below in the ingress NPU on 0/1 manages the queues of local ports like et-0/1/0 and remote ports like et-0/2/0. Same thing for the ingress structure of the NPU(s) on 0/2 handling the queues for remote port et-0/1/0 and local port et-0/2/0.

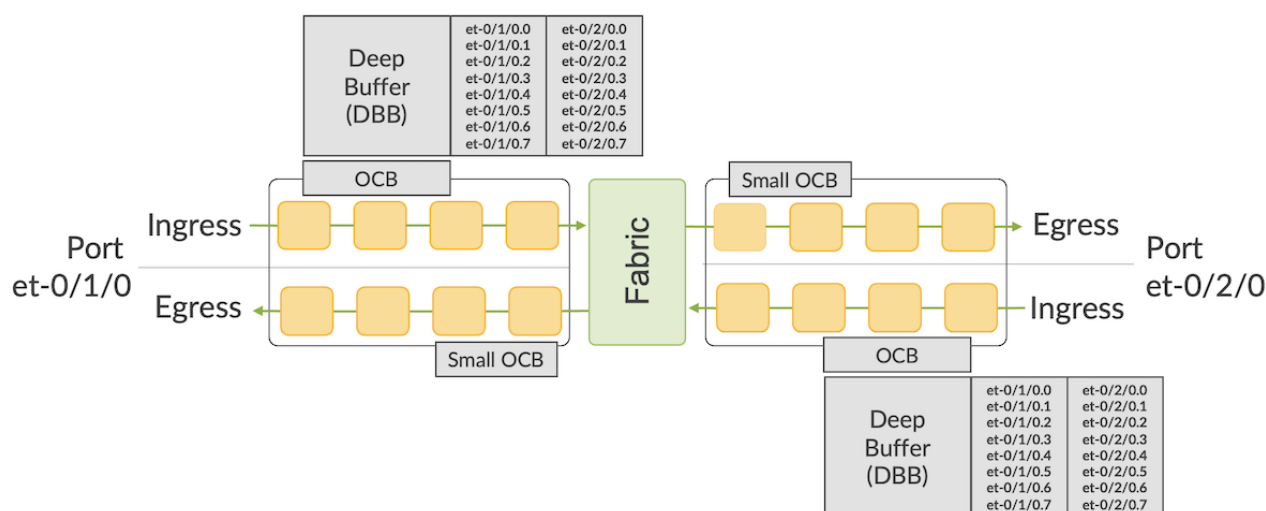


Figure 3: VOQ for two ports

The treatment is the same if the packet is targeted to the same interface it comes from (loopback or routing to a different VLAN) or if it will be sent to a remote port.

We can go further with the idea, if we have 1,000 egress ports or attachment points, we will need to create 1,000 sets of 8 queues (total 8,000 queues) on all PFEs present in the router. If it's a modular chassis made of 24 PFEs, I will need to create this 8,000-queue structure in all 24 ASICs.

That's why, by default, a VOQ scale should be considered a global number. If your ASIC supports 64,000 VOQs, it means the entire router supports 64,000 queues and all egress ports will need to share this pool. It's a fundamental difference compared to systems where packets are stored in ingress and egress paths, like the Trio chipset in the MX Series.

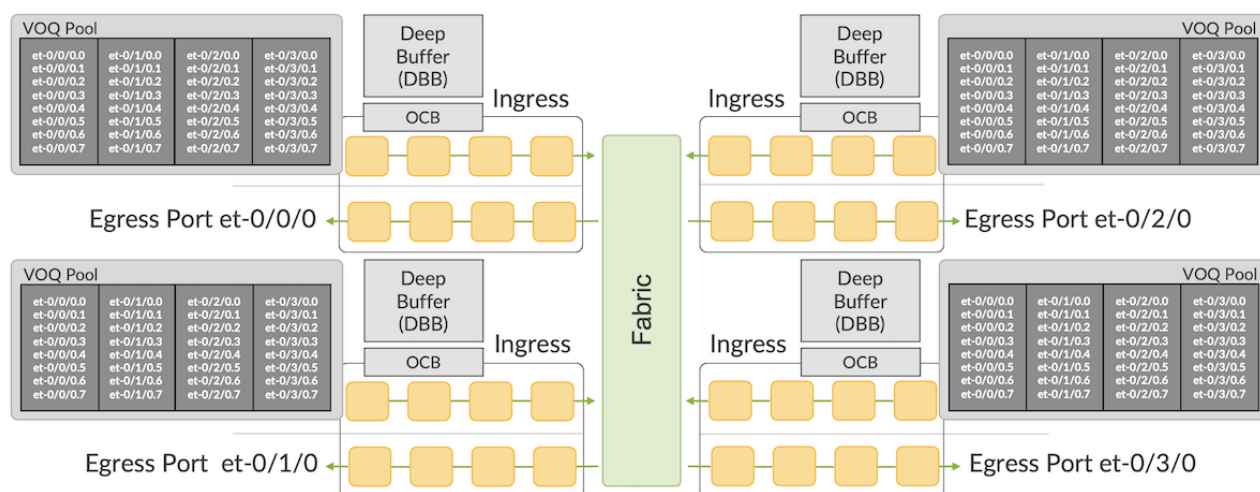


Figure 4: example with 4 PFEs and 4 egress interfaces

To illustrate it, Diagram 4 above represents 4 different egress interfaces located on 4 distinct PFEs. You can extend the idea to thousands of ports (it's just pretty hard to illustrate on a diagram ;))

1.4 Life of a Packet

In the ingress pipeline, we will break the process into four steps:

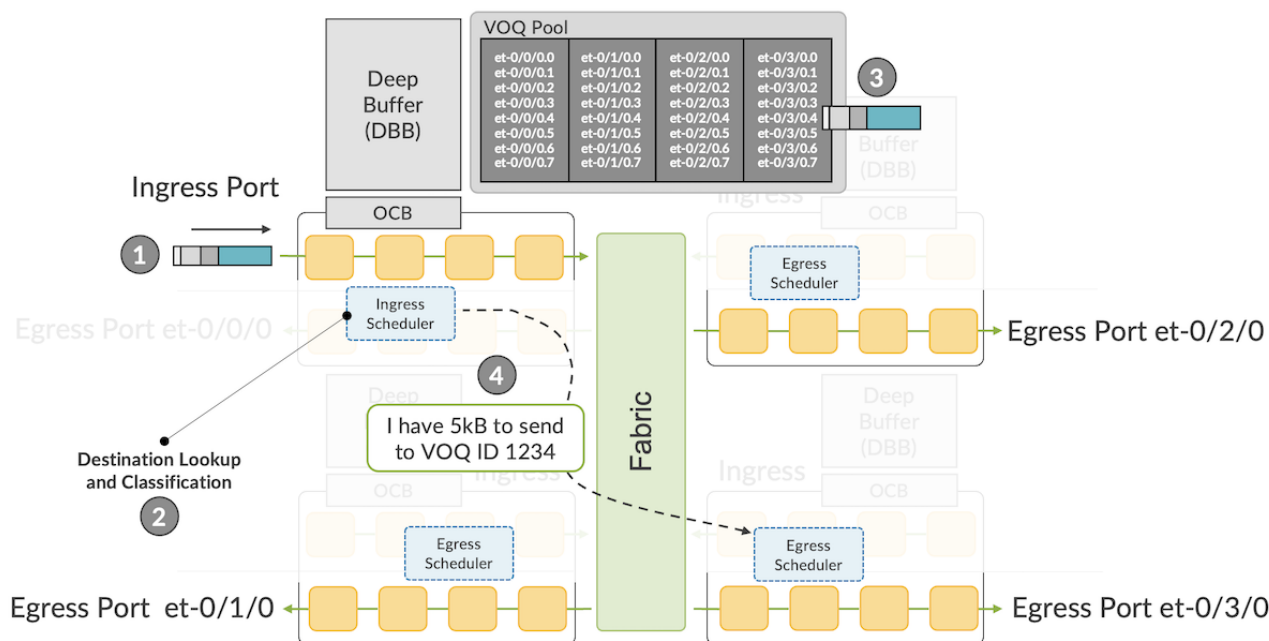


Figure 5: Life of a packet, Ingress

1. A packet is received on port et-0/0/0 (mapped to the core 0 of the PFE 0/0)
2. In this ingress pipeline, multiple blocks will treat the packet, determining where the packet is meant to be sent to, and which class of traffic it belongs to. This will help figure out an egress port (and the egress PFE associated) and a queue, represented by a VOQ ID. In our example, port et-0/3/0 and queue 5, VOQ 1234
3. The packet is stored in HBM, with additional internal headers, including information like the VOQ ID. The system keeps track of the memory position used to store this packet.
4. The Ingress Scheduler informs the destination Egress Scheduler present in the PFE pipeline associated with the port et-0/3/0 that it has 5kB of traffic for this particular VOQ. And that's all it will do now until further notification.

For the egress part of the process, let's imagine our Egress Scheduler on PFE 0/3 received solicitations to transmit packets to interface et-0/3/0 from 3 different ingress PFEs.

From PFE 0/2 (top right of the diagram), it received a solicitation for a high-priority packet in port et-0/3/0 queue 7 (VOQ ID 3445).

And from PFE 0/0 and 0/1 it received a solicitation for packets targeted to port et-0/3/0 queue 4, the best effort queue (VOQ ID 1234).

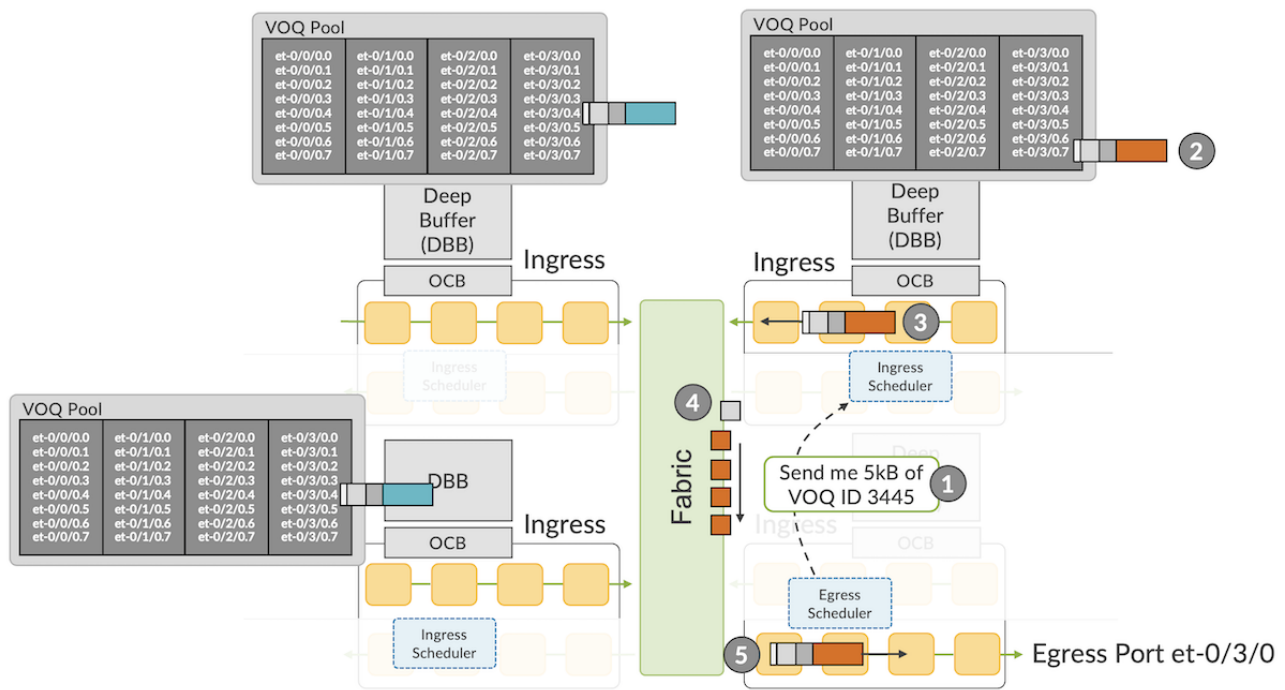


Figure 6: Life of a packet, Egress

Our egress PFE has 3 solicitations to address and based on the QoS configuration, it will decide that the high-priority packet from PFE 0/2 needs to be transmitted.

1. The Egress Scheduler will generate a token or grant, to the Ingress Scheduler of PFE 0/2 (top right), for a specific amount of traffic.
2. With this “right to transmit” the Ingress Scheduler will trigger the dequeue of this high-priority packet
3. It will continue its journey in the last blocks of the ingress pipeline. The last step will consist in splitting the packets into cells and spraying them among the available fabric interfaces
4. The cells are routed to the destination PFE in the fabric
5. Cells are re-assembled and the high-priority packet continues its journey in the egress pipeline, with a short buffering in the “EGQ” (Egress port Queue). In that step, a very basic classification will occur, differentiating high from low priority packets and unicast from multicast. After all egress features are applied, the packet is transmitted to its next-hop via et-0/3/0

The system is now ready to service the two remaining solicitations. Since they have the same destination and queue, they have the same priority and will be treated in a round-robin fashion. The same 5 steps described above will be executed, a token sent to the Ingress Scheduler, the packets “cell-ified” will travel to the destination PFE via the fabric interfaces, and they will be eventually transmitted to port et-0/3/0.

In this model, the packet forwarding is “scheduled”, which means it relies entirely on the constant and extremely fast communication between all ingress and egress schedulers of the different PFEs in the system.

Note: the total traffic to port et-0/3/0 queue 4 is not really measured on the egress PFE, but it’s the sum of all the requests to transmit scattered among all ingress PFEs. That makes the traffic accounting a bit more challenging in very large multi-PFE systems.

It also applies to system-on-the-chip (SoC) architectures, even if they are made of a single core (like the Jericho2c for example).

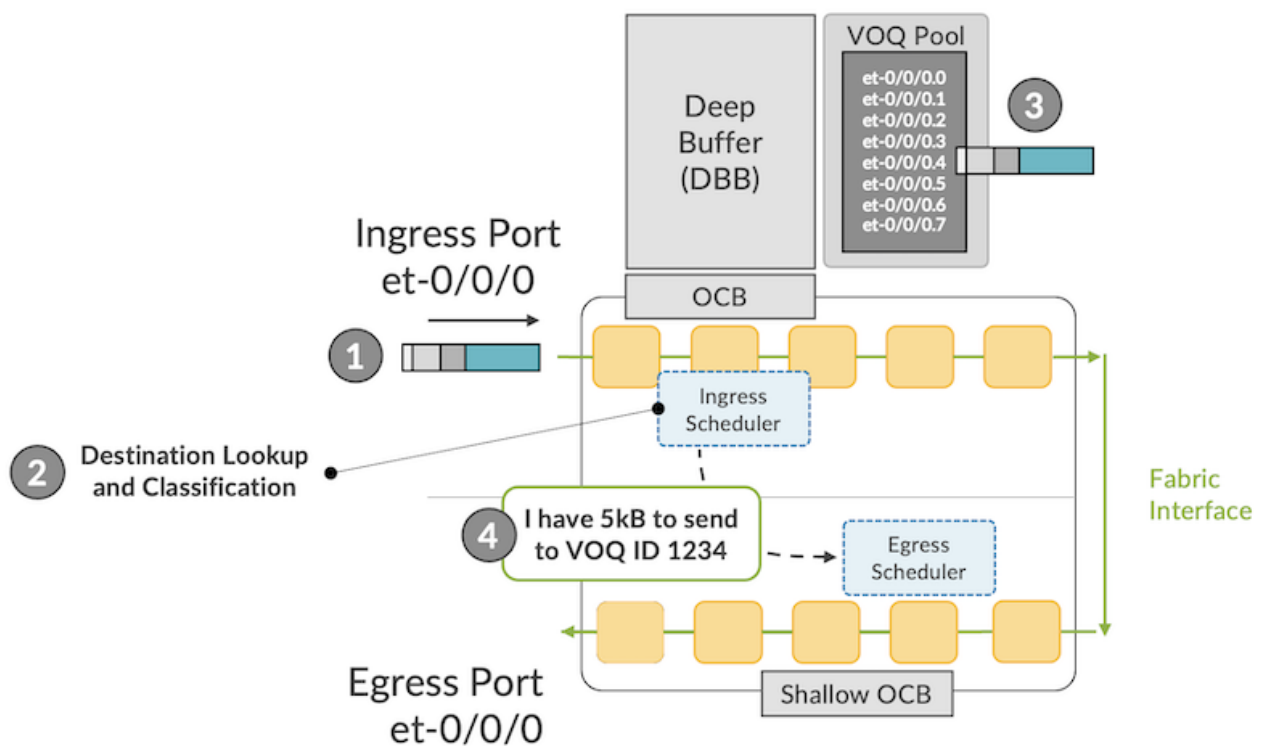


Figure 7: Life of a packet in SoC architecture, Ingress

In the ingress direction, we have the same steps as described earlier, with the only difference, the destination resolution points to a local interface. But whether the egress pipeline is local or remote, it makes no difference from the ingress scheduler's perspective, it simply informs the destination of its desire to transmit traffic to a specific VOQ.

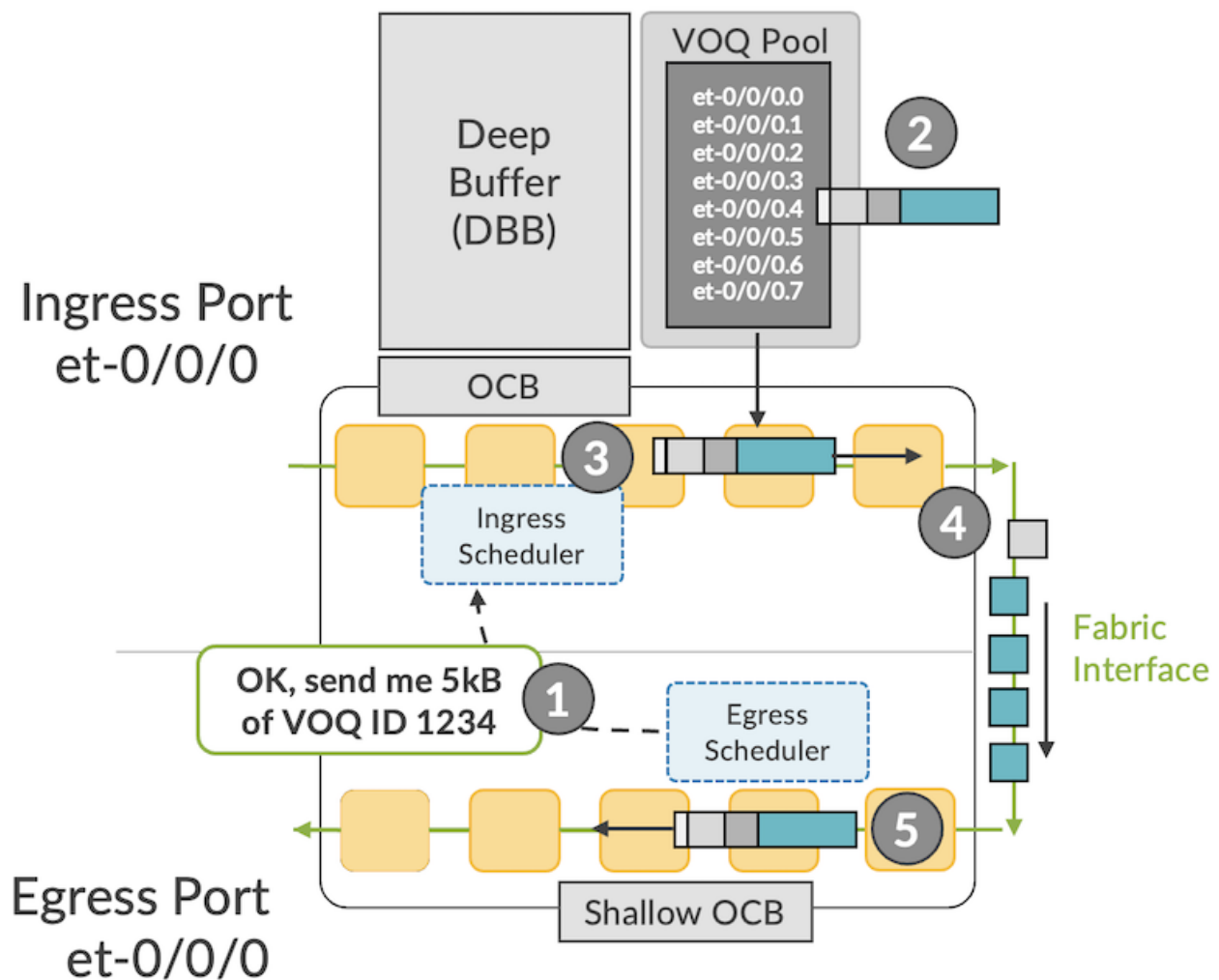


Figure 8: Life of a packet in SoC architecture, Egress

On the egress side too, no difference if the request went from a distant or a local scheduler. It will decide the allocation of tokens/rights to transmit based on the quality of service configuration. Even if everything happens in the same PFE core, the packets are still “cell-ified” to be transmitted to the fabric interface, and the cells are re-assembled when arriving in the egress pipeline.

1.5 Illustration with CLI

You’ll understand now the key concept: the queues for a specific egress port are scattered all around the router. Refer to this article to understand the different Packet Forwarding Engines (PFE) used in the different ACX7k family members: <https://juniper.github.io/techposts/building-the-acx7000-series-the-pfe/article>

If it’s a single PFE device (aka SoC/RoC for System/Router on a Chip), the queues are naturally present on this single NPU, as it’s the case for an ACX7100-32C or ACX7100-48L:

```
1 regress@rtme-acx7100-32c-a> show interfaces et-0/0/0 | grep Index
2   Interface index: 1006, SNMP ifIndex: 505
3   Logical interface et-0/0/0.16386 (Index 1001) (SNMP ifIndex 508)
4
5 regress@rtme-acx7100-32c-a> show interfaces et-0/0/1 | grep Index
6   Interface index: 1007, SNMP ifIndex: 507
7   Logical interface et-0/0/1.16386 (Index 1002) (SNMP ifIndex 510)
```

```
8
9 regress@rtme-acx7100-32c-a> show interfaces et-0/0/2 | grep Index
10 Interface index: 1008, SNMP ifIndex: 509
11 Logical interface et-0/0/2.16386 (Index 1003) (SNMP ifIndex 512)
12
13 regress@rtme-acx7100-32c-a> show interfaces et-0/0/3 | grep Index
14 Interface index: 1009, SNMP ifIndex: 511
15 Logical interface et-0/0/3.16386 (Index 1004) (SNMP ifIndex 514)
16
17 jnx(/dev/pts/1)# show cos ifd 1006
18 IFD Name : et-0/0/0
19 IFD Index : 1006
20 BCM Port : 1
21 IFD h/w information
22 slotId : 0
23 Unit : 0
24 BCM Port : 1
25 System Port : 1
26 Port Gport : 0x8000001
27 Voq Base id : 40
28 Unit : 0 Connector Flow id : 40
29 Unit : 0 Gport ucast voq connector group : 0xc4000028
30 Port Speed : 100000000000 bps
31 Shaping Rate : 0 bps
32 AE scale factor: 1
33 Scheduler map Id : 2
34
35 jnx(/dev/pts/1)# show cos ifd 1007
36 IFD Name : et-0/0/1
37 IFD Index : 1007
38 BCM Port : 3
39 IFD h/w information
40 slotId : 0
41 Unit : 0
42 BCM Port : 3
43 System Port : 3
44 Port Gport : 0x8000003
45 Voq Base id : 56
46 Unit : 0 Connector Flow id : 56
47 Unit : 0 Gport ucast voq connector group : 0xc4000038
48 Port Speed : 100000000000 bps
49 Shaping Rate : 0 bps
50 AE scale factor: 1
51 Scheduler map Id : 2
52
53 jnx(/dev/pts/1)# show cos ifd 1008
54 IFD Name : et-0/0/2
55 IFD Index : 1008
56 BCM Port : 5
57 IFD h/w information
58 slotId : 0
59 Unit : 0
60 BCM Port : 5
61 System Port : 5
62 Port Gport : 0x8000005
63 Voq Base id : 72
64 Unit : 0 Connector Flow id : 72
65 Unit : 0 Gport ucast voq connector group : 0xc4000048
66 Port Speed : 100000000000 bps
67 Shaping Rate : 0 bps
68 AE scale factor: 1
69 Scheduler map Id : 2
70
71 jnx(/dev/pts/1)# show cos ifd 1009
72 IFD Name : et-0/0/3
73 IFD Index : 1009
74 BCM Port : 7
75 IFD h/w information
76 slotId : 0
77 Unit : 0
78 BCM Port : 7
79 System Port : 7
80 Port Gport : 0x8000007
81 Voq Base id : 88
82 Unit : 0 Connector Flow id : 88
83 Unit : 0 Gport ucast voq connector group : 0xc4000058
84 Port Speed : 100000000000 bps
85 Shaping Rate : 0 bps
86 AE scale factor: 1
87 Scheduler map Id : 2
88
```

```
89 jnx(/dev/pts/1)#
```

In the example of ACX7100-32C above, we consider the first four 100GE ports.

The VOQ Base ID represents the first VOQ ID for an interface. The system allocates 8 VOQ IDs per interface, but you'll notice with have 16 between et-0/0/0 and et-0/0/1 and 32 between et-0/0/0 and et-0/0/2...

It's simply because port et-0/0/0 can be used with a breakout cable and four ports. It will disable the port et-0/0/1 so we will have 4 channelized interfaces from et-0/0/0:0 to et-0/0/0:3, each one will be allocated 8 VOQs.

In the next example, we are considering an ACX7509. It's a centralized platform based on two REs and two FEBs. Each forwarding board contains two Qumran2C in a back-to-back configuration.

Logically, half of the ports will be connected on one PFE and the other half to the second PFE.

```
1 root@acx7509-plb-a-re0> show interfaces queue et-6/0/0
2 Physical interface: et-6/0/0, up, Physical link is Up
3 Interface index: 1026, SNMP ifIndex: 505
4 Forwarding classes: 8 supported, 8 in use
5 Egress queues: 8 supported, 8 in use
6 Queue: 0, Forwarding classes: q0
7   Queued:
8     Packets : 0 0 pps
9     Bytes : 0 0 bps
10  Transmitted:
11    Packets : 0 0 pps
12    Bytes : 0 0 bps
13    Tail-dropped packets : 0 0 pps
14    Tail-dropped bytes : 0 0 bps
15    RED-dropped packets : 0 0 pps
16    RED-dropped bytes : 0 0 bps
17 Queue: 1, Forwarding classes: q1
18   Queued:
19     Packets : 0 0 pps
20     Bytes : 0 0 bps
21  Transmitted:
22    Packets : 0 0 pps
23    Bytes : 0 0 bps
24    Tail-dropped packets : 0 0 pps
25    Tail-dropped bytes : 0 0 bps
26    RED-dropped packets : 0 0 pps
27    RED-dropped bytes : 0 0 bps
28 Queue: 2, Forwarding classes: q2
29   Queued:
30     Packets : 0 0 pps
31     Bytes : 0 0 bps
32  Transmitted:
33    Packets : 0 0 pps
34    Bytes : 0 0 bps
35    Tail-dropped packets : 0 0 pps
36    Tail-dropped bytes : 0 0 bps
37    RED-dropped packets : 0 0 pps
38    RED-dropped bytes : 0 0 bps
39 Queue: 3, Forwarding classes: q3
40   Queued:
41     Packets : 72660 0 pps
42     Bytes : 7629300 152 bps
43  Transmitted:
44    Packets : 72660 0 pps
45    Bytes : 7629300 152 bps
46    Tail-dropped packets : 0 0 pps
47    Tail-dropped bytes : 0 0 bps
48    RED-dropped packets : 0 0 pps
49    RED-dropped bytes : 0 0 bps
50 Queue: 4, Forwarding classes: q4
51   Queued:
52     Packets : 0 0 pps
53     Bytes : 0 0 bps
54  Transmitted:
55    Packets : 0 0 pps
56    Bytes : 0 0 bps
57    Tail-dropped packets : 0 0 pps
58    Tail-dropped bytes : 0 0 bps
59    RED-dropped packets : 0 0 pps
```

```
60 RED-dropped bytes : 0 0 bps
61 Queue: 5, Forwarding classes: q5
62   Queued:
63     Packets : 0 0 pps
64     Bytes : 0 0 bps
65   Transmitted:
66     Packets : 0 0 pps
67     Bytes : 0 0 bps
68   Tail-dropped packets : 0 0 pps
69   Tail-dropped bytes : 0 0 bps
70   RED-dropped packets : 0 0 pps
71   RED-dropped bytes : 0 0 bps
72 Queue: 6, Forwarding classes: q6
73   Queued:
74     Packets : 0 0 pps
75     Bytes : 0 0 bps
76   Transmitted:
77     Packets : 0 0 pps
78     Bytes : 0 0 bps
79   Tail-dropped packets : 0 0 pps
80   Tail-dropped bytes : 0 0 bps
81   RED-dropped packets : 0 0 pps
82   RED-dropped bytes : 0 0 bps
83 Queue: 7, Forwarding classes: q7
84   Queued:
85     Packets : 0 0 pps
86     Bytes : 0 0 bps
87   Transmitted:
88     Packets : 0 0 pps
89     Bytes : 0 0 bps
90   Tail-dropped packets : 0 0 pps
91   Tail-dropped bytes : 0 0 bps
92   RED-dropped packets : 0 0 pps
93   RED-dropped bytes : 0 0 bps
94
95 {master}
96 root@acx7509-p1b-a-re0>
97
98 root@acx7509-p1b-a-re0> show interfaces voq et-6/0/0
99 Physical interface: et-6/0/0, up, Physical link is Up
100 Interface index: 1026, SNMP ifIndex: 505
101
102 Queue: 0, Forwarding classes: q0
103
104 FPC number: 0
105 PFE: 0
106   Queued:
107     Packets : 0 0 pps
108     Bytes : 0 0 bps
109   Transmitted:
110     Packets : 0 0 pps
111     Bytes : 0 0 bps
112   Dropped:
113     Tail-dropped packets : 0 0 pps
114     Tail-dropped bytes : 0 0 bps
115     RED-dropped packets : 0 0 pps
116     RED-dropped bytes : 0 0 bps
117   Queue-depth bytes:
118     Current : 0
119     Peak : 0
120     Average : 0
121
122 PFE: 1
123   Queued:
124     Packets : 0 0 pps
125     Bytes : 0 0 bps
126   Transmitted:
127     Packets : 0 0 pps
128     Bytes : 0 0 bps
129   Dropped:
130     Tail-dropped packets : 0 0 pps
131     Tail-dropped bytes : 0 0 bps
132     RED-dropped packets : 0 0 pps
133     RED-dropped bytes : 0 0 bps
134   Queue-depth bytes:
135     Current : 0
136     Peak : 0
137     Average : 0
138
139
140 Queue: 1, Forwarding classes: q1
```

```
141
142 FPC number: 0
143 PFE: 0
144 Queued:
145   Packets : 0 0 pps
146   Bytes : 0 0 bps
147 Transmitted:
148   Packets : 0 0 pps
149   Bytes : 0 0 bps
150 Dropped:
151   Tail-dropped packets : 0 0 pps
152   Tail-dropped bytes : 0 0 bps
153   RED-dropped packets : 0 0 pps
154   RED-dropped bytes : 0 0 bps
155 Queue-depth bytes:
156   Current : 0
157   Peak : 0
158   Average : 0
159
160 PFE: 1
161 Queued:
162   Packets : 0 0 pps
163   Bytes : 0 0 bps
164 Transmitted:
165   Packets : 0 0 pps
166   Bytes : 0 0 bps
167 Dropped:
168   Tail-dropped packets : 0 0 pps
169   Tail-dropped bytes : 0 0 bps
170   RED-dropped packets : 0 0 pps
171   RED-dropped bytes : 0 0 bps
172 Queue-depth bytes:
173   Current : 0
174   Peak : 0
175   Average : 0
176
177
178 Queue: 2, Forwarding classes: q2
179
180 FPC number: 0
181 PFE: 0
182 Queued:
183   Packets : 0 0 pps
184   Bytes : 0 0 bps
185 Transmitted:
186   Packets : 0 0 pps
187   Bytes : 0 0 bps
188 Dropped:
189   Tail-dropped packets : 0 0 pps
190   Tail-dropped bytes : 0 0 bps
191   RED-dropped packets : 0 0 pps
192   RED-dropped bytes : 0 0 bps
193 Queue-depth bytes:
194   Current : 0
195   Peak : 0
196   Average : 0
197
198 PFE: 1
199 Queued:
200   Packets : 0 0 pps
201   Bytes : 0 0 bps
202 Transmitted:
203   Packets : 0 0 pps
204   Bytes : 0 0 bps
205 Dropped:
206   Tail-dropped packets : 0 0 pps
207   Tail-dropped bytes : 0 0 bps
208   RED-dropped packets : 0 0 pps
209   RED-dropped bytes : 0 0 bps
210 Queue-depth bytes:
211   Current : 0
212   Peak : 0
213   Average : 0
214
215
216 Queue: 3, Forwarding classes: q3
217
218 FPC number: 0
219 PFE: 0
220 Queued:
221   Packets : 0 0 pps
```

```
222     Bytes : 0 0 bps
223     Transmitted:
224     Packets : 0 0 pps
225     Bytes : 0 0 bps
226     Dropped:
227     Tail-dropped packets : 0 0 pps
228     Tail-dropped bytes : 0 0 bps
229     RED-dropped packets : 0 0 pps
230     RED-dropped bytes : 0 0 bps
231     Queue-depth bytes:
232     Current : 0
233     Peak : 0
234     Average : 0
235
236     PFE: 1
237     Queued:
238     Packets : 72666 0 pps
239     Bytes : 7629930 152 bps
240     Transmitted:
241     Packets : 72666 0 pps
242     Bytes : 7629930 152 bps
243     Dropped:
244     Tail-dropped packets : 0 0 pps
245     Tail-dropped bytes : 0 0 bps
246     RED-dropped packets : 0 0 pps
247     RED-dropped bytes : 0 0 bps
248     Queue-depth bytes:
249     Current : 0
250     Peak : 0
251     Average : 0
252
253
254 Queue: 4, Forwarding classes: q4
255
256     FPC number: 0
257     PFE: 0
258     Queued:
259     Packets : 0 0 pps
260     Bytes : 0 0 bps
261     Transmitted:
262     Packets : 0 0 pps
263     Bytes : 0 0 bps
264     Dropped:
265     Tail-dropped packets : 0 0 pps
266     Tail-dropped bytes : 0 0 bps
267     RED-dropped packets : 0 0 pps
268     RED-dropped bytes : 0 0 bps
269     Queue-depth bytes:
270     Current : 0
271     Peak : 0
272     Average : 0
273
274     PFE: 1
275     Queued:
276     Packets : 0 0 pps
277     Bytes : 0 0 bps
278     Transmitted:
279     Packets : 0 0 pps
280     Bytes : 0 0 bps
281     Dropped:
282     Tail-dropped packets : 0 0 pps
283     Tail-dropped bytes : 0 0 bps
284     RED-dropped packets : 0 0 pps
285     RED-dropped bytes : 0 0 bps
286     Queue-depth bytes:
287     Current : 0
288     Peak : 0
289     Average : 0
290
291
292 Queue: 5, Forwarding classes: q5
293
294     FPC number: 0
295     PFE: 0
296     Queued:
297     Packets : 0 0 pps
298     Bytes : 0 0 bps
299     Transmitted:
300     Packets : 0 0 pps
301     Bytes : 0 0 bps
302     Dropped:
```

```
303     Tail-dropped packets : 0 0 pps
304     Tail-dropped bytes : 0 0 bps
305     RED-dropped packets : 0 0 pps
306     RED-dropped bytes : 0 0 bps
307     Queue-depth bytes:
308         Current : 0
309         Peak : 0
310         Average : 0
311
312     PFE: 1
313     Queued:
314         Packets : 0 0 pps
315         Bytes : 0 0 bps
316     Transmitted:
317         Packets : 0 0 pps
318         Bytes : 0 0 bps
319     Dropped:
320         Tail-dropped packets : 0 0 pps
321         Tail-dropped bytes : 0 0 bps
322         RED-dropped packets : 0 0 pps
323         RED-dropped bytes : 0 0 bps
324     Queue-depth bytes:
325         Current : 0
326         Peak : 0
327         Average : 0
328
329
330 Queue: 6, Forwarding classes: q6
331
332     FPC number: 0
333     PFE: 0
334     Queued:
335         Packets : 0 0 pps
336         Bytes : 0 0 bps
337     Transmitted:
338         Packets : 0 0 pps
339         Bytes : 0 0 bps
340     Dropped:
341         Tail-dropped packets : 0 0 pps
342         Tail-dropped bytes : 0 0 bps
343         RED-dropped packets : 0 0 pps
344         RED-dropped bytes : 0 0 bps
345     Queue-depth bytes:
346         Current : 0
347         Peak : 0
348         Average : 0
349
350     PFE: 1
351     Queued:
352         Packets : 0 0 pps
353         Bytes : 0 0 bps
354     Transmitted:
355         Packets : 0 0 pps
356         Bytes : 0 0 bps
357     Dropped:
358         Tail-dropped packets : 0 0 pps
359         Tail-dropped bytes : 0 0 bps
360         RED-dropped packets : 0 0 pps
361         RED-dropped bytes : 0 0 bps
362     Queue-depth bytes:
363         Current : 0
364         Peak : 0
365         Average : 0
366
367
368 Queue: 7, Forwarding classes: q7
369
370     FPC number: 0
371     PFE: 0
372     Queued:
373         Packets : 0 0 pps
374         Bytes : 0 0 bps
375     Transmitted:
376         Packets : 0 0 pps
377         Bytes : 0 0 bps
378     Dropped:
379         Tail-dropped packets : 0 0 pps
380         Tail-dropped bytes : 0 0 bps
381         RED-dropped packets : 0 0 pps
382         RED-dropped bytes : 0 0 bps
383     Queue-depth bytes:
```

```

384     Current : 0
385     Peak : 0
386     Average : 0
387
388     PFE: 1
389     Queued:
390         Packets : 0 0 pps
391         Bytes : 0 0 bps
392     Transmitted:
393         Packets : 0 0 pps
394         Bytes : 0 0 bps
395     Dropped:
396         Tail-dropped packets : 0 0 pps
397         Tail-dropped bytes : 0 0 bps
398         RED-dropped packets : 0 0 pps
399         RED-dropped bytes : 0 0 bps
400     Queue-depth bytes:
401         Current : 0
402         Peak : 0
403         Average : 0
404
405 {master}
406 root@acx7509-plb-a-re0>

```

You notice in these CLI outputs that we can display the aggregated stats in the show interface queue while we display the stats of the VOQ for each individual PFE with the show interface voq command.

In this ACX7509 case above, we just have two PFEs.

But the output will become much longer with a chassis form factor like the ACX7908. In this last example below, we have 3 line cards (FPC):

- Slot 2: FPC with a single PFE
- Slot 3: same kind of FPC with a single PFE
- Slot 6: denser FPC with 3 PFEs

```

1 root@acx7908-sw-pfe-d-re0> show interfaces queue et-6/0/9:0
2 Physical interface: et-6/0/9:0, up, Physical link is Up
3   Interface index: 1216, SNMP ifIndex: 810
4   Forwarding classes: 8 supported, 4 in use
5   Egress queues: 8 supported, 4 in use
6   Queue: 0, Forwarding classes: best-effort
7   Queued:
8       Packets : 0 0 pps
9       Bytes : 0 0 bps
10  Transmitted:
11      Packets : 0 0 pps
12      Bytes : 0 0 bps
13      Tail-dropped packets : 0 0 pps
14      Tail-dropped bytes : 0 0 bps
15      RED-dropped packets : 0 0 pps
16      RED-dropped bytes : 0 0 bps
17  Queue: 1, Forwarding classes: expedited-forwarding
18  Queued:
19      Packets : 0 0 pps
20      Bytes : 0 0 bps
21  Transmitted:
22      Packets : 0 0 pps
23      Bytes : 0 0 bps
24      Tail-dropped packets : 0 0 pps
25      Tail-dropped bytes : 0 0 bps
26      RED-dropped packets : 0 0 pps
27      RED-dropped bytes : 0 0 bps
28  Queue: 2, Forwarding classes: assured-forwarding
29  Queued:
30      Packets : 0 0 pps
31      Bytes : 0 0 bps
32  Transmitted:
33      Packets : 0 0 pps
34      Bytes : 0 0 bps
35      Tail-dropped packets : 0 0 pps
36      Tail-dropped bytes : 0 0 bps
37      RED-dropped packets : 0 0 pps

```

```
38 RED-dropped bytes : 0 0 bps
39 Queue: 3, Forwarding classes: network-control
40 Queued:
41   Packets : 0 0 pps
42   Bytes : 0 0 bps
43 Transmitted:
44   Packets : 0 0 pps
45   Bytes : 0 0 bps
46 Tail-dropped packets : 0 0 pps
47 Tail-dropped bytes : 0 0 bps
48 RED-dropped packets : 0 0 pps
49 RED-dropped bytes : 0 0 bps
50
51 {master}
52 root@acx7908-sw-pfe-d-re0> show interfaces voq et-6/0/9:0
53 Physical interface: et-6/0/9:0, up, Physical link is Up
54 Interface index: 1216, SNMP ifIndex: 810
55
56 Queue: 0, Forwarding classes: best-effort
57
58 FPC number: 2
59 PFE: 0
60 Queued:
61   Packets : 0 0 pps
62   Bytes : 0 0 bps
63 Transmitted:
64   Packets : 0 0 pps
65   Bytes : 0 0 bps
66 Dropped:
67   Tail-dropped packets : 0 0 pps
68   Tail-dropped bytes : 0 0 bps
69   RED-dropped packets : 0 0 pps
70   RED-dropped bytes : 0 0 bps
71 Queue-depth bytes:
72   Current : 0
73   Peak : 0
74   Average : 0
75
76
77 FPC number: 3
78 PFE: 0
79 Queued:
80   Packets : 0 0 pps
81   Bytes : 0 0 bps
82 Transmitted:
83   Packets : 0 0 pps
84   Bytes : 0 0 bps
85 Dropped:
86   Tail-dropped packets : 0 0 pps
87   Tail-dropped bytes : 0 0 bps
88   RED-dropped packets : 0 0 pps
89   RED-dropped bytes : 0 0 bps
90 Queue-depth bytes:
91   Current : 0
92   Peak : 0
93   Average : 0
94
95
96 FPC number: 6
97 PFE: 0
98 Queued:
99   Packets : 0 0 pps
100   Bytes : 0 0 bps
101 Transmitted:
102   Packets : 0 0 pps
103   Bytes : 0 0 bps
104 Dropped:
105   Tail-dropped packets : 0 0 pps
106   Tail-dropped bytes : 0 0 bps
107   RED-dropped packets : 0 0 pps
108   RED-dropped bytes : 0 0 bps
109 Queue-depth bytes:
110   Current : 0
111   Peak : 0
112   Average : 0
113
114 PFE: 1
115 Queued:
116   Packets : 0 0 pps
117   Bytes : 0 0 bps
118 Transmitted:
```

```

119   Packets : 0 0 pps
120   Bytes : 0 0 bps
121   Dropped:
122     Tail-dropped packets : 0 0 pps
123     Tail-dropped bytes : 0 0 bps
124     RED-dropped packets : 0 0 pps
125     RED-dropped bytes : 0 0 bps
126   Queue-depth bytes:
127     Current : 0
128     Peak : 0
129     Average : 0
130
131   PFE: 2
132   Queued:
133     Packets : 0 0 pps
134     Bytes : 0 0 bps
135   Transmitted:
136     Packets : 0 0 pps
137     Bytes : 0 0 bps
138   Dropped:
139     Tail-dropped packets : 0 0 pps
140     Tail-dropped bytes : 0 0 bps
141     RED-dropped packets : 0 0 pps
142     RED-dropped bytes : 0 0 bps
143   Queue-depth bytes:
144     Current : 0
145     Peak : 0
146     Average : 0
147
148   Queue: 1, Forwarding classes: expedited-forwarding
149
150   <We removed the output for the three remaining queues for the sake of brevity>
151
152   {master}
153   root@acx7908-sw-pfe-d-re0>

```

1.6 The pipeline

You can find a lot of details in the Traffic Management Architecture Design Guide here: <https://docs.broadcom.com/doc/88800-DG1-PUB>

To summarize the key concepts, we can represent the pipeline like this:

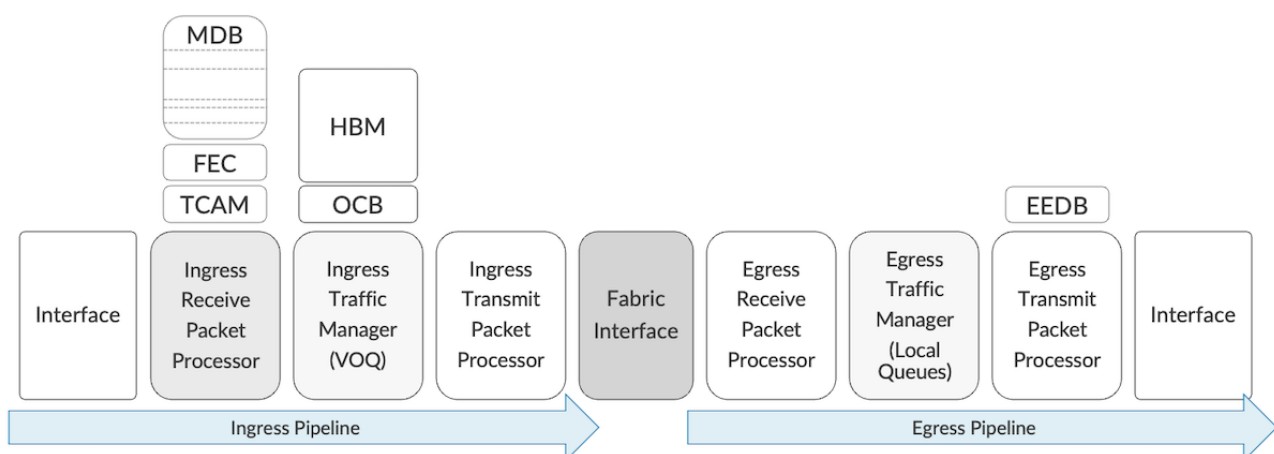


Figure 9: High-level view of the DNX Pipeline

Packets are received from ingress interfaces, but this description doesn't only apply to network interfaces (NIFs). Traffic could also come from the:

- Recycle interfaces

- CPU interface
- OAMP (Operation and Maintenance Processor) interface
- SAT (Service Availability Testing) interface
- And some others like OLP (Offload Processor) interfaces

The first 144 bytes of each packet are sent to the IRPP (Ingress Receive Packet Processor).

- IRPP takes care of the majority of the features, including the identification of the incoming interface, the lookup (L2, L3, MPLS, ...), the next hop resolution and load balancing, the filtering, the insertion of internal headers and many more
- ITM handles the queues, on-chip or off-chip. That's also where the multicast replication can be performed
- ITPP is the last step before the fabric interface, packets can be edited here also
- ERPP manages some of the egress features like filtering
- ETM handles the queues from the egress perspective and can also perform multicast replication
- ETPP can edit packets based on indication from the ingress pipeline (present in the internal headers)

1.7 Conclusion

This article provided some details on the life of a unicast packet in the DNX-based systems and presented the concept of Virtual Output Queue, essential to understand how the ACX7k Series products operate. In the next articles, we will dig deep into the internals of each ACX7k family members.

1.8 Useful links

- <https://docs.broadcom.com/doc/88800-DG1-PUB>
- Building the ACX7000 Series - The PFE: <https://juniper.github.io/techposts/building-the-acx7000-series-the-pfe/article>

1.9 Glossary

- CoS/QoS: Class of Service / Quality of Service
- DBB: Delay Bandwidth Buffer
- DDR: Double Data Rate (Memory)
- EGQ: Egress Queue
- HBM: High Bandwidth Memory
- IFL: Logical Interface
- PFE: Packet Forwarding Engine
- NPU: Network Processing Unit
- OAMP: Operation and Maintenance Processor
- OCB: On-Chip Buffer
- OLP: Offload Processor
- SAT/ Service Availability Testing
- TM: Traffic Manager
- VOQ / VOQ ID: Virtual Output Queue / Identifier

- SoC: System on a Chip

1.10 Acknowledgements

Thanks a lot to Biju Kumar Karunakaran Nair for all the details on queueing and review of this document.



DC – AI – Routing – Switching – Security – Wireless – Automation

© Copyright 2026 Hewlett Packard Enterprise Development LP. The information contained herein is subject to change without notice. The only warranties for Hewlett Packard Enterprise products and services are set forth in the express warranty statements accompanying such products and services. Nothing herein should be construed as constituting an additional warranty.

Hewlett Packard Enterprise shall not be liable for technical or editorial errors or omissions contained herein.

Trademark acknowledgments, if needed. All third-party marks are the property of their respective owners.

HEWLETT PACKARD ENTERPRISE

HPE.com

HPE

